Cloud & DevOps Capstone

Handoff Guide v3: build the project from scratch

Purpose. This document lets any AI assistant (ChatGPT, DeepSeek, or another Claude account) continue helping the learner from exactly where the previous assistant stopped. Read sections 1 to 4 first, follow the teaching rules in section 1, then work through the roadmap in section 6 one step at a time. A ready-to-paste starter prompt is on the last page.

Status right now: planning is finished and **nothing has been built yet**. The learner asked not to start building until he has moved to the new assistant. The next action is Step 1 (write and test the Flask app).

1. About the learner and how to teach him

- **Name:** Atif (full name Abdur Rahman Naseer Mohammed Khan). B.E. Computer Science graduate, Hyderabad, India. Goal: become job-ready in Cloud and DevOps.
- **Already practiced:** AWS (EC2, S3, VPC, ALB, ASG, CloudFormation, Config, EventBridge, Lambda, IAM), Docker (nginx and Flask images, Docker Hub), Kubernetes fundamentals (Minikube, kubeadm, Pods, Nodes, scaling), Terraform basics, Python (Flask, boto3), bash, Git. He has built a WhatsApp notification before in a CSPM project.
- **Has NOT covered:** PostgreSQL. Does not know **Helm**. So the project must not need a database, and the application must not use Helm.
- **Environment:** Windows laptop with Git Bash (MINGW64). Ubuntu for EC2. Default AWS region us-east-1. Personal resource prefix **ARNMK** (for example ARNMK-capstone-ec2, ARNMK-capstone-sg): keep it in all AWS resource names.
- **Learning platforms:** KodeKloud and Killerkoda.

Teaching rules (firm preferences)

- **Concept first, always.** Explain what it is, why it is needed, how it works, and a real-world analogy before any commands. Keep concept sections short and high-signal, but never skip them.
- **One step at a time.** Give a single step, wait for the learner to run it and confirm the result, then continue. He often shares screenshots of errors: read them precisely and give targeted fixes.
- **Commands must be short, exact and copy-paste-ready.** Explain what each command does, the expected output, and common mistakes.
- **Stay inside the course syllabus.** No extra tools. If something looks borderline, ask the learner whether the course covered it before using it.
- **Preferred structure per topic:** What is it? / Why needed? / How it works? / Real-world analogy / Architecture-Workflow / Practical implementation / Best practices / Common mistakes / Interview questions / Summary.
- **He must be able to explain every line** because the final evaluation includes live technical questions. Write the files together, one at a time, explaining each part. Do not just dump finished code.

2. The assignment (tutor's capstone briefing)

Title: Cloud-Native Application Deployment, CI/CD Automation and Observability. Individual, hands-on project. Screenshots, recorded video or theory alone do not count. Everything must work in a live demo.

Requirement	What must be shown
Documentation	Application idea (problem, purpose, features, users), architecture diagram, deployment architecture, CI/CD architecture, notification flow, monitoring architecture. Explain each component and how they interact.
GitHub + CI/CD	GitHub repo with source and config. Pipeline with a **push trigger** that builds the app and a container image.
Docker Hub	Built images pushed to a Docker Hub repository.
Notification	A successful pipeline run sends a message to Slack, Teams or **WhatsApp** (learner chose WhatsApp).
Deployment	Kubernetes cluster (or approved serverless). Application reachable and working.
Monitoring	Prometheus + Grafana. Dashboard with CPU and memory, node and cluster resources, pod status, pod resource use, restarts, workload metrics, **HPA metrics and replica counts**.
HPA live demo	Show the app working, explain the HPA (target metric, min and max replicas), run a stress test, show the HPA detecting load, pods scaling out automatically, Grafana showing it, then stop the load and watch scale-down. Manual replica changes do not count.
Final presentation	Must include a **fresh code push**, automated image build and push, pipeline notification, and HPA scaling under load. Be ready to explain design decisions and troubleshoot.
Technical note	HPA on CPU needs **Metrics Server**. Prometheus and Grafana alone do not enable HPA.

Evaluation criteria: documentation and architecture, CI/CD automation, deployment and functionality, monitoring and observability, autoscaling demonstration, technical understanding, presentation and live demo.

3. Decisions already made

- **Build from scratch.** The learner writes his own small Python Flask application so he can explain every line. A public repo (github.com/rkawasthiii/devops-project, MediDesk) was reviewed earlier. It is **not** used as the base because it needs PostgreSQL, Helm, Terraform and EKS. Do not copy it; ideas worth borrowing are /health probes, tagging images with the commit SHA, a load-test script, and a deliberate troubleshooting exercise.
- **No database.** Appointments are kept in memory. No PostgreSQL.
- **No Helm anywhere.** Plain Kubernetes YAML files (Namespace, Deployment, Service, HPA).
- **Notification:** WhatsApp (Twilio WhatsApp Sandbox recommended, see section 7).
- **Where it runs:** one Ubuntu EC2 instance (t3.large) in us-east-1 running single-node Minikube.
- **CI/CD tool:** GitHub Actions (confirmed: the course used it). Jenkins is not needed and would add load on the small EC2 instance; only use it if the tutor insists.
- **Monitoring install method (confirmed):** the course installed Prometheus and Grafana from **web downloads** (standalone programs). So both run as ordinary programs on the EC2 host, outside the cluster, and reach into Minikube through **kubectl proxy**. See Step 7.

4. Do our own app and setup meet every requirement?

Yes. Mapping each briefing item to what we build:

Briefing item	How the from-scratch project satisfies it
Documentation + diagrams	Learner writes a README and draws two diagrams (application architecture, CI/CD pipeline) in draw.io.
GitHub repository	Repo contains app.py, requirements.txt, Dockerfile, k8s/ manifests, monitoring/ files, and .github/workflows/ci.yml.
Push-triggered pipeline	ci.yml with on: push to main. Builds the image on every push.

Briefing item	How the from-scratch project satisfies it
Docker Hub	Pipeline pushes DOCKERHUB_USERNAME/clinic-app with tags latest and the commit SHA.
Notification	Last pipeline step sends a WhatsApp message through the Twilio API, only when the job succeeded.
Kubernetes deployment	Minikube on EC2. Deployment (2 replicas), Service, Namespace, all as plain YAML.
CPU and memory, node and pod metrics, restarts, pod status	Prometheus (downloaded binary on the EC2 host) scrapes cAdvisor (kubelet, container CPU and memory), kube-state-metrics (pod status, restarts, HPA) and node_exporter (host CPU and memory). Shown in Grafana (downloaded binary).
Application or workload metrics	The Flask app exposes /metrics (prometheus_client): request count, latency histogram, appointments booked.
HPA metrics and replica counts	kube-state-metrics exposes HPA current, desired, min and max replicas. Shown in Grafana.
HPA live demo	App has a deliberately CPU-heavy /work endpoint. Busybox load pods hit it. HPA (CPU target 50 percent, min 2, max 6) scales out automatically, and back in after the load stops.
Fresh code push in the demo	Learner changes the version text in app.py, pushes, Actions builds and pushes the image and sends WhatsApp, rollout restarts the Deployment, the app now shows the new version.

Honest caveats to explain to evaluators:

- Data is stored in memory, so each pod has its own list of appointments and restarts lose data. That is a deliberate simplification (no database in the syllabus). A real system would use a database.
- Deploying the new image to Minikube after a push: the briefing's demo flow needs the updated app verified, not necessarily auto-deployed. The simple approach is a manual kubectl rollout restart after the WhatsApp message arrives. A self-hosted GitHub runner on the EC2 instance would make it fully automatic, but it is an extra; only add it if the course covered it.
- The Twilio WhatsApp sandbox has limits (see section 7). Test it the day before the demo.
- Because Prometheus reaches the app through the Kubernetes Service, each scrape lands on one random pod, so the app-level panels (request rate, latency) are approximate when there are several pods. The CPU, memory, pod status, restart and HPA replica panels, which matter most for the demo, are not affected.

5. Target architecture

```
Developer --git push--> GitHub repo
                            |
                      GitHub Actions (on push to main)
                        1. checkout code   2. docker build
                        3. push to Docker Hub (tags: latest + commit SHA)
                        4. send WhatsApp message (only on success)
                            |
                       Docker Hub  <------ image pulled by the cluster
                            |
AWS EC2 host (Ubuntu t3.large, us-east-1)   [ARNMK-capstone-ec2]
  Programs on the host (from web downloads):
    node_exporter :9100   Prometheus :9090   Grafana :3000   kubectl proxy :8001
  Minikube (single-node Kubernetes, Docker driver)
    namespace clinic:       clinic-app Deployment (2-6 pods, HPA) + Service
    kube-system:            Metrics Server (feeds the HPA), kube-state-metrics

Prometheus scrapes: node_exporter directly, and through kubectl proxy:
  cAdvisor (kubelet), kube-state-metrics, clinic-app /metrics
Grafana queries Prometheus (dashboard).

Browser --> EC2 public IP:8080 (app), :3000 (Grafana), :9090 (Prometheus)
Load test: busybox pods inside the cluster call the /work endpoint repeatedly
```

6. Roadmap: core steps (do one at a time)

For each step: short concept explanation, then the commands, then the expected output, then wait for the learner's confirmation. Reference code for the files is in Appendix A; teach it line by line as the learner writes it.

Step 0. Prerequisites

- Accounts: GitHub, Docker Hub, AWS, Twilio (for WhatsApp).
- Laptop: Git Bash, Docker Desktop, a code editor. Python 3 is useful for a quick local test but optional, because the app runs inside Docker (the EC2 instance does not need Python at all).

Step 1. Write and test the Flask app locally

Concept: a small web service with endpoints for the app (/, /appointments), health checks (/health), metrics (/metrics) and a CPU stress endpoint (/work). Create a project folder (for example clinic-app) with app.py and requirements.txt from Appendix A. Run it with Python or go straight to Docker in Step 2.

```
pip install -r requirements.txt
python app.py
curl http://localhost:5000/health
curl -X POST http://localhost:5000/appointments \
  -H "Content-Type: application/json" \
  -d '{"patient":"Ravi","doctor":"Dr Rao","time":"10:30"}'
curl http://localhost:5000/appointments
curl http://localhost:5000/metrics
```

Step 2. Containerize and push to Docker Hub by hand

Concept: a Dockerfile packages the app so it runs the same everywhere. Create the Dockerfile from Appendix A, then build, run, test and push once manually. Replace DOCKERHUB_USERNAME with the learner's Docker Hub username (ask him for it).

```
docker build -t clinic-app:v1 .
docker run -p 5000:5000 clinic-app:v1
docker login
docker tag clinic-app:v1 DOCKERHUB_USERNAME/clinic-app:v1
docker push DOCKERHUB_USERNAME/clinic-app:v1
```

Step 3. GitHub repo and the CI/CD pipeline with WhatsApp

- Create a GitHub repo and push the project. Add the workflow file .github/workflows/ci.yml from Appendix A.
- In the repo: Settings, Secrets and variables, Actions. Add secrets DOCKERHUB_USERNAME, DOCKERHUB_TOKEN (a Docker Hub access token, not the password), TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_WHATSAPP_FROM, TWILIO_WHATSAPP_TO. WhatsApp setup is in section 7.
- Push a small change. Expected: green Actions run, new tags in Docker Hub, WhatsApp message received.

Step 4. Launch the EC2 instance and install Minikube

- AWS Console, region us-east-1. Ubuntu 22.04 or 24.04, **t3.large** (2 vCPU, 8 GB RAM), 30 GB gp3, new key pair. Name tag ARNMK-capstone-ec2.
- Security group ARNMK-capstone-sg: inbound 22 (SSH), 8080 (app), 3000 (Grafana), 9090 (Prometheus, optional), source My IP only.
- Stop the instance when not working to save cost. The public IP changes after stop and start.

```
sudo apt update && sudo apt install -y docker.io git curl conntrack
sudo usermod -aG docker $USER && newgrp docker
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
sudo snap install kubectl --classic
minikube start --driver=docker --cpus=2 --memory=4500
kubectl get nodes
```

Expected: one node named minikube in Ready state. Memory is set to 4500 MB so the host keeps about 3.5 GB for Prometheus, Grafana and node_exporter.

Step 5. Deploy the app with plain manifests

Concept: a Namespace groups resources, a Deployment keeps the desired number of pods running, a Service gives them one stable address. Create the three files in k8s/ from Appendix A (put the learner's Docker Hub username in the image field). The Deployment must have CPU **requests**, otherwise the HPA cannot compute a percentage.

```
git clone https://github.com/GITHUB_USERNAME/clinic-app.git
cd clinic-app
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml
kubectl get pods -n clinic
nohup kubectl port-forward --address 0.0.0.0 svc/clinic-svc 8080:5000 -n clinic > pf-app.log 2>&1 &
```

Open http://EC2-PUBLIC-IP:8080 in a browser (use the real public IP). Expected: 2 pods Running and JSON from the home page. Replace GITHUB_USERNAME with the learner's username.

Step 6. Metrics Server and the HPA

Concept: the HPA checks pod CPU every ~15 seconds and adds or removes pods to keep average utilization near the target. It reads CPU from Metrics Server.

```
minikube addons enable metrics-server
kubectl top nodes
kubectl top pods -n clinic
kubectl apply -f k8s/hpa.yaml
kubectl get hpa -n clinic
```

Expected: TARGETS shows something like 2%/50%, not <unknown>. If unknown: wait one to two minutes for Metrics Server, and confirm the Deployment has CPU requests.

Step 7. Prometheus and Grafana (downloaded programs on the EC2 host)

Concept: Prometheus **scrapes** (pulls) metrics from HTTP endpoints on a schedule and stores them as time series; Grafana queries Prometheus and draws dashboards. As in the course, both are downloaded from their websites and run as normal programs on the EC2 machine. They reach inside Minikube through **kubectl proxy**, a local gateway to the cluster that handles authentication. Four sources feed Prometheus: node_exporter (host CPU and memory), cAdvisor (container CPU and memory, built into the kubelet), kube-state-metrics (pod status, restarts and HPA replica counts; it is the only source for those) and the app's own /metrics. kube-state-metrics is a small program that runs as a pod and is installed with plain YAML, no Helm.

7a. kube-state-metrics (in the cluster) and kubectl proxy

```
git clone https://github.com/kubernetes/kube-state-metrics.git
kubectl apply -f kube-state-metrics/examples/standard
kubectl get pods -n kube-system
nohup kubectl proxy --port=8001 > proxy.log 2>&1 &
curl -s localhost:8001/api/v1/nodes/minikube/proxy/metrics/cadvisor | head -5
```

Expected: a kube-state-metrics pod Running, and lines of metric text from the curl. If the node is not called minikube, use the name from kubectl get nodes.

7b. node_exporter

Open prometheus.io/download, find node_exporter for linux amd64, and copy the link of the tar.gz file (do not guess a version number).

```
wget LINK_COPIED_FROM_THE_DOWNLOAD_PAGE
tar xvf node_exporter-*.tar.gz
cd node_exporter-*/
nohup ./node_exporter > node.log 2>&1 &
curl -s localhost:9100/metrics | head -5
cd ..
```

7c. Prometheus

On the same prometheus.io/download page copy the link for Prometheus (linux amd64). Replace the default prometheus.yml inside the extracted folder with the one in Appendix A, then start it.

```
wget LINK_COPIED_FROM_THE_DOWNLOAD_PAGE
tar xvf prometheus-*.tar.gz
cd prometheus-*/
nano prometheus.yml
nohup ./prometheus --config.file=prometheus.yml > prom.log 2>&1 &
cd ..
```

Open http://EC2-PUBLIC-IP:9090, Status, Targets. Expected: all five jobs UP (prometheus, node, clinic-app, kube-state-metrics, cadvisor). In nano: paste the file, Ctrl+O, Enter, Ctrl+X.

7d. Grafana

Open grafana.com/grafana/download, choose OSS, Linux, standalone binaries, and copy the tar.gz link.

```
wget LINK_COPIED_FROM_THE_DOWNLOAD_PAGE
tar xvf grafana-*.tar.gz
cd grafana-*/
nohup ./bin/grafana server > graf.log 2>&1 &
cd ..
```

Open http://EC2-PUBLIC-IP:3000, log in as admin with password admin (set a new one). Connections, Data sources, add Prometheus with URL http://localhost:9090, Save and test. If the command grafana server is not found, the older binary is ./bin/grafana-server.

7e. After a stop and start of the instance

Nothing restarts by itself. Use the start script in Appendix A (start-all.sh) to bring everything back, and remember the public IP will have changed.

Step 8. Build the Grafana dashboard

One dashboard, every panel explained in the presentation. Check metric names in Prometheus first and adjust labels.

Panel	PromQL
Pod CPU usage	sum(rate(container_cpu_usage_seconds_total{namespace="clinic",container!=""}[2m])) by (pod)
Pod memory usage	sum(container_memory_working_set_bytes{namespace="clinic",container!=""}) by (pod)
Pod status	sum(kube_pod_status_phase{namespace="clinic"}) by (phase)
Pod restarts	sum(kube_pod_container_status_restarts_total{namespace="clinic"}) by (pod)
HPA replicas (current, desired)	kube_horizontalpodautoscaler_status_current_replicas{namespace="clinic"} and kube_horizontalpodautoscaler_status_desired_replicas{namespace="clinic"}
HPA min and max	kube_horizontalpodautoscaler_spec_min_replicas and kube_horizontalpodautoscaler_spec_max_replicas
Node CPU %	100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100)
Node memory %	(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100
Request rate (app, approximate)	sum(rate(clinic_http_requests_total[1m]))
p95 latency (app)	histogram_quantile(0.95, sum(rate(clinic_request_latency_seconds_bucket[2m])) by (le))
Appointments booked	clinic_appointments_booked_total

Step 9. HPA load test rehearsal

```
kubectl get hpa -n clinic -w
kubectl get pods -n clinic -w
kubectl run load1 --image=busybox:1.36 --restart=Never -n clinic -- \
```

```
  /bin/sh -c "while true; do wget -q -O- http://clinic-svc:5000/work; done"
kubectl run load2 --image=busybox:1.36 --restart=Never -n clinic -- \
  /bin/sh -c "while true; do wget -q -O- http://clinic-svc:5000/work; done"
kubectl delete pod load1 load2 -n clinic
```

- Run the two watch commands in separate SSH terminals with Grafana open. Start the load pods; within a minute or two CPU rises above 50 percent and replicas climb toward 6. Delete the load pods and watch scale-down (the HPA manifest sets a 60 second scale-down window so the demo is shorter).
- If nothing scales: confirm the load pods are Running, check kubectl top pods, and confirm the Service name and port in the wget URL.

Step 10. Documentation and final rehearsal

- Write the documentation and draw the two diagrams (outline in section 9).
- Start everything first (start-all.sh). Rehearse the full flow twice: change the VERSION string in app.py, push, watch Actions, receive the WhatsApp message, run kubectl rollout restart deployment/clinic-app -n clinic, refresh the app to show the new version, run the load test, show the HPA scaling and Grafana, stop the load, show scale-down.

7. WhatsApp notification setup

Recommended: **Twilio WhatsApp Sandbox** (free to try). The learner built WhatsApp notifications before in his CSPM project; if he used another provider there, reuse it. Slack or Teams webhooks are also allowed by the briefing and are simpler if WhatsApp gives trouble.

- Create a Twilio account, open the WhatsApp Sandbox in the console, and from his WhatsApp send the word join followed by the code shown there, to the sandbox number.
- TWILIO_WHATSAPP_FROM is the sandbox number shown in the console (for example +14155238886). TWILIO_WHATSAPP_TO is his own number with country code.
- The sandbox has session limits: re-join about every 72 hours, and free-form messages work within a time window after you last messaged the sandbox. Send the join message again shortly before the demo and test the pipeline first.

8. Troubleshooting cheat sheet

Symptom	Likely cause and first check
ImagePullBackOff	Wrong image name or tag, or a private Docker Hub repo. kubectl describe pod, read Events.
HPA shows unknown	Metrics Server not ready, or no CPU requests on the Deployment. kubectl top pods; check the Deployment resources.
Pods Pending	Not enough CPU or memory on the node. kubectl describe pod. Use t3.large; restart Minikube with enough memory.
No scaling during load test	Load pods not Running, wrong URL, or CPU request too high. Check kubectl top pods while load runs.
Prometheus target down	kubectl proxy not running, wrong node name or Service name in prometheus.yml, or kube-state-metrics pod not Running. Test the same path with curl on localhost:8001. The app Service port must be named http.
Grafana or Prometheus not reachable	Program not running (check with ps aux and its log file), or port 3000 or 9090 missing from the security group, or the public IP changed.
Cannot open app in browser	Security group missing the port, public IP changed after restart, or port-forward not started with --address 0.0.0.0 (or it died; restart it).
Docker Hub push fails in Actions	Wrong secret name, expired token, or image name missing the username prefix.

Symptom	Likely cause and first check
No WhatsApp message	Sandbox join expired, wrong number format, or empty secrets. Check Twilio message logs and the Actions log.
Service has no endpoints	Selector labels do not match pod labels. kubectl get endpoints -n clinic and kubectl get pods --show-labels -n clinic.
Old version still showing after push	Deployment not restarted, or image cached. Run kubectl rollout restart deployment/clinic-app -n clinic (imagePullPolicy Always pulls the new latest).

9. Deliverables checklist and documentation outline

- [ ] Project documentation [ ] Application architecture diagram [ ] CI/CD pipeline diagram
- [ ] GitHub repository [ ] Docker Hub repository with images [ ] Working push-triggered pipeline
- [ ] WhatsApp notification on success [ ] Application deployed on Kubernetes
- [ ] Prometheus and Grafana with the dashboard [ ] HPA configured and demonstrated under load
- [ ] Final presentation with a rehearsed live demo

Documentation outline

- Application idea: clinic appointment booking; problem, purpose, features, intended users (reception staff).
- Architecture and deployment diagram (EC2, Minikube, namespaces, Service, pods).
- CI/CD architecture: push, Actions, build, Docker Hub, WhatsApp, deployment.
- Notification system: how the pipeline calls the WhatsApp API.
- Monitoring architecture: what is scraped, where it is stored, what each Grafana panel means and how it shows resource pressure and scaling.
- HPA configuration: CPU target 50 percent, min 2, max 6, how scaling decisions are made.
- Design decisions (in-memory data, plain YAML, single-node Minikube) and troubleshooting notes.

10. Open questions to confirm with the learner

- Already answered: the course used GitHub Actions, and Prometheus and Grafana were installed from web downloads.
- What is his Docker Hub username and GitHub username? (Needed for the image names.)
- Has he used the Twilio sandbox, or a different WhatsApp provider, in his CSPM project?
- How many days remain before the demo? This decides how much rehearsal time there is.

Appendix A. Reference code (teach each part, do not just paste)

app.py

```
import time, math
from flask import Flask, request, jsonify
from prometheus_client import Counter, Histogram, generate_latest, CONTENT_TYPE_LATEST

app = Flask(__name__)
VERSION = "v1"
appointments = []

REQUESTS = Counter("clinic_http_requests_total", "Total HTTP requests", ["method", "endpoint"])
LATENCY = Histogram("clinic_request_latency_seconds", "Request latency", ["endpoint"])
BOOKINGS = Counter("clinic_appointments_booked_total", "Appointments booked")

@app.before_request
def start_timer():
    request.start_time = time.time()

@app.after_request
def record(resp):
    REQUESTS.labels(request.method, request.path).inc()
    LATENCY.labels(request.path).observe(time.time() - request.start_time)
    return resp

@app.route("/")
def home():
    return jsonify(app="Clinic Appointments", version=VERSION, total=len(appointments))

@app.route("/appointments", methods=["GET"])
def list_appointments():
    return jsonify(appointments)

@app.route("/appointments", methods=["POST"])
def book():
    data = request.get_json(force=True)
    appt = {"id": len(appointments) + 1, "patient": data.get("patient"),
            "doctor": data.get("doctor"), "time": data.get("time")}
    appointments.append(appt)
    BOOKINGS.inc()
    return jsonify(appt), 201

@app.route("/health")
def health():
    return jsonify(status="ok")

@app.route("/work")
def work():
    n = 0.0
    for i in range(2000000):
        n += math.sqrt(i)
    return jsonify(result=n)

@app.route("/metrics")
def metrics():
    return generate_latest(), 200, {"Content-Type": CONTENT_TYPE_LATEST}

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt and Dockerfile

```
flask
prometheus-client

FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

k8s/namespace.yaml and k8s/service.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: clinic
---
apiVersion: v1
kind: Service
metadata:
  name: clinic-svc
  namespace: clinic
  labels:
    app: clinic-app
spec:
  selector:
    app: clinic-app
  ports:
  - name: http
    port: 5000
    targetPort: 5000
```

k8s/deployment.yaml (replace DOCKERHUB_USERNAME)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: clinic-app
  namespace: clinic
  labels:
    app: clinic-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: clinic-app
  template:
    metadata:
      labels:
        app: clinic-app
    spec:
      containers:
      - name: clinic-app
        image: DOCKERHUB_USERNAME/clinic-app:latest
        imagePullPolicy: Always
        ports:
        - containerPort: 5000
        resources:
          requests:
            cpu: 100m
            memory: 64Mi
          limits:
            cpu: 500m
            memory: 256Mi
        readinessProbe:
          httpGet:
            path: /health
            port: 5000
        livenessProbe:
          httpGet:
            path: /health
            port: 5000
```

k8s/hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: clinic-hpa
  namespace: clinic
spec:
```

```
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: clinic-app
  minReplicas: 2
  maxReplicas: 6
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 60
```

prometheus.yml (Step 7c)

```
global:
  scrape_interval: 15s
scrape_configs:
- job_name: prometheus
  static_configs:
  - targets: ["localhost:9090"]
- job_name: node
  static_configs:
  - targets: ["localhost:9100"]
- job_name: clinic-app
  metrics_path: /api/v1/namespaces/clinic/services/clinic-svc:http/proxy/metrics
  static_configs:
  - targets: ["localhost:8001"]
- job_name: kube-state-metrics
  metrics_path: /api/v1/namespaces/kube-system/services/kube-state-metrics:http-metrics/proxy/metrics
  static_configs:
  - targets: ["localhost:8001"]
- job_name: cadvisor
  metrics_path: /api/v1/nodes/minikube/proxy/metrics/cadvisor
  static_configs:
  - targets: ["localhost:8001"]
```

start-all.sh (run from the home folder after the instance restarts; adjust folder names to what was extracted)

```
#!/bin/bash
minikube start
nohup kubectl proxy --port=8001 > proxy.log 2>&1 &
cd ~/node_exporter-*/ && nohup ./node_exporter > node.log 2>&1 &
cd ~/prometheus-*/ && nohup ./prometheus --config.file=prometheus.yml > prom.log 2>&1 &
cd ~/grafana-*/ && nohup ./bin/grafana server > graf.log 2>&1 &
nohup kubectl port-forward --address 0.0.0.0 svc/clinic-svc 8080:5000 -n clinic > pf-app.log 2>&1 &
```

.github/workflows/ci.yml

```
name: ci
on:
  push:
    branches: [main]
jobs:
  build-push-notify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
```

```
        ${{ secrets.DOCKERHUB_USERNAME }}/clinic-app:latest
        ${{ secrets.DOCKERHUB_USERNAME }}/clinic-app:${{ github.sha }}
    - name: Notify WhatsApp
      if: success()
      env:
        SID: ${{ secrets.TWILIO_ACCOUNT_SID }}
        TOKEN: ${{ secrets.TWILIO_AUTH_TOKEN }}
      run: |
        curl -s -X POST \
          "https://api.twilio.com/2010-04-01/Accounts/$SID/Messages.json" \
          --data-urlencode "From=whatsapp:${{ secrets.TWILIO_WHATSAPP_FROM }}" \
          --data-urlencode "To=whatsapp:${{ secrets.TWILIO_WHATSAPP_TO }}" \
          --data-urlencode "Body=Pipeline succeeded. Image pushed to Docker Hub." \
          -u "$SID:$TOKEN"
```

These files were drafted from knowledge, not tested on a live cluster. Verify each one while running it and fix anything that differs (for example metric names in Prometheus, or service names).

Starter prompt to paste into the next AI

Upload or paste this whole document first, then send:

```
You are my Cloud Computing and DevOps learning assistant. I am Atif, a beginner
working on a college capstone. The attached document has the briefing, my
decisions, the architecture, the roadmap and reference code. Please:
1. Read it fully and confirm in 5 lines what the project is and where we are.
2. Follow the teaching rules in section 1: concept first, one step at a time,
   short copy-paste commands, stay within my syllabus (I have not learned
   PostgreSQL or Helm; my course installed Prometheus and Grafana from web
   downloads, and used GitHub Actions).
3. Ask me the remaining open questions in section 10, one at a time.
4. Then start Step 1: explain the concept, write app.py with me, and wait for
   my confirmation before the next step.
5. When I share an error or screenshot, diagnose it precisely and give a fix.
6. If anything in the document looks wrong, tell me and verify before going on.
```

Tip: keep a running log of finished steps, errors and fixes in a text file. When you switch assistants, paste the log together with this document.